

James Smith

Mechatronics Engineering Student

Waterloo, ON | JamesThomasSmith07@icloud.com | [GitHub](#) | [LinkedIn](#) | <https://james-t-smith.github.io/>



Based in Waterloo, ON.

- Currently open to Winter 2027 internships

Professional Projects

CAD to CV model suite

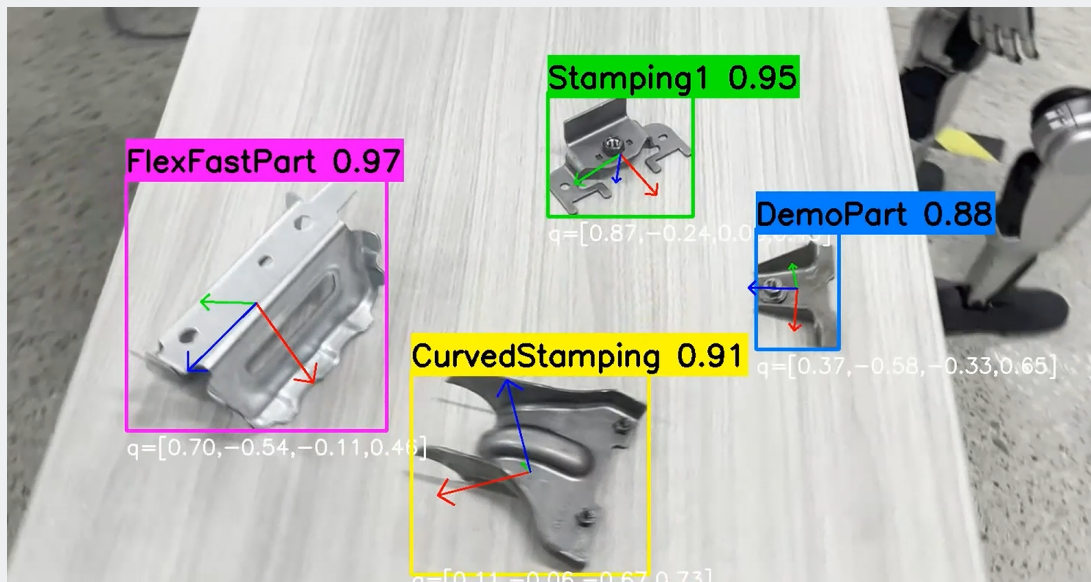
Synthetic Dataset Generator and Computer Vision Model Training Suite that created detector

DATES May 2026 – Aug 2026

TAGS Dataset Engineering, Computer Vision, Neural Network Training

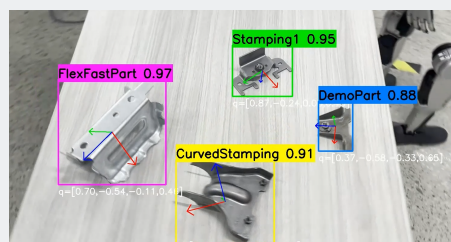
TOOLS Open CV, RESNET, Ultralytics YOLO, Pytorch

This project includes a synthetic dataset generator able to create thousands of unique, annotated training images of specific parts from cad models in minutes. A smart dataloader able to train Ultralytics YOLO CV models on randomized data sets based on a learning curve optimizing curriculum. It can create a pair of neural networks, a YOLO based bounding box detector, and a RESNET based 3D angular orientation regression model.



Four Class Detections

0% real images trained or annotated for this model, pre filtered raw detections

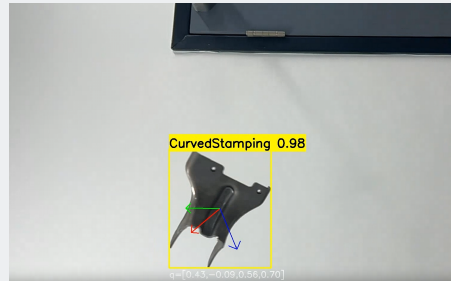


Test of Both Detection and Orientation Model on Real Frames

0% real images trained for this model, pre filtered raw detections

[View on website](#)

The roposity was a modular tool for rapidly creating and exporting computer vision models custom to company parts using only a CAD file of the part. It allowed the user to upload CAD files and start the automatic generation/training procedure within minutes, completely bypassing the hours of capturing and annotating training images. It was able to create models that preformed better, overall, and on edge cases, than hand trained alternatives.

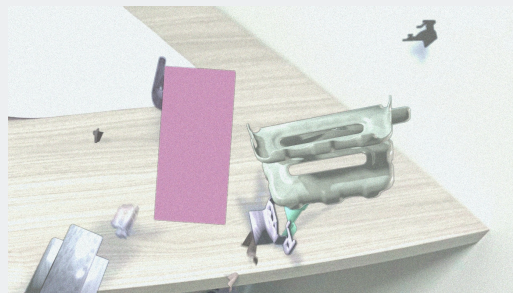


Real Time Detection and Orientation Regression

f

[View on website](#)

Using PyRender to overlay parts ontop of background images, thousands of unique, annotated training images could be created instantly, completely removing the tedious, hours long process of annotating images for computer vision model training.

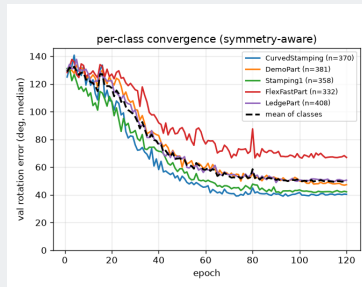


Example Training Rendering

Randomized training image, can be generated in seconds with annotation.

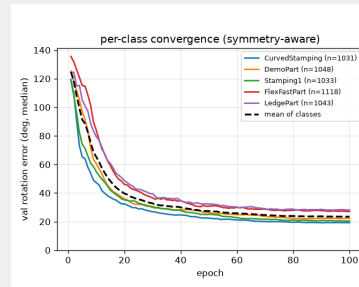
Datasets could be generated using many randomizing features to strengthen the models edge case preformance. Textures, color, lighting, shadows, blur, size, orientation, and visibility could all be randomized per part, and scenes could be blurred, noized, cropped, and augmented for extra randomization. Distractors were also randomized to prevent false positive detections and the model learning on static sceness

EFFECT OF CIRRICULUM ON MODEL TRAINING



Without Cirriculum

C.



With Cirriculum

Includes.

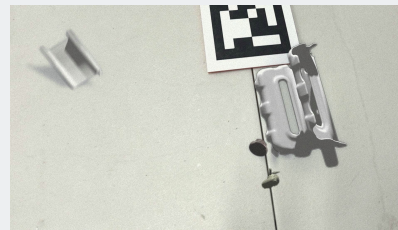
Implamenting a smart dataloader increased both the speed and accuracy of training the orientation model. The cirriculum was deloped by calculating an overall abstraction score for each image based on the randomized values that made up the image. Images with many annotated parts, high contrast, low blur, and less distractors were given a high abstraction score, and were trained in earlier epochs. This allowed the model to converge rapidly, within the first quarter of training the model has a good understanding of the parts it is looking for. Alternatively, images with many distractors, fewer parts, low contrast, and more blur/distortion were given a low abstraction score, and were fed during the later portion of training. These "realer" images fortified the model, to not just understand the features of the part, but to be able to infer when parts were less visible, or blurred. They also enforced which features were not detections using distractors to prevent false positives.

ABSTRACTED RENDERING VS REAL RENDERING



High Abstraction Rendering

Many parts that allow fast learning.



Low Abstraction Rendering

Simulates actual deployment images reinforcing model
bahavior.

PROJECT OUTCOME

Models that were generated can be used for any application that requires 6D pose of a part. Applications could include part inspection, line validation/part tracking, typical robotics and humanoid robotics.

Colaborative Robot Convayer Loader

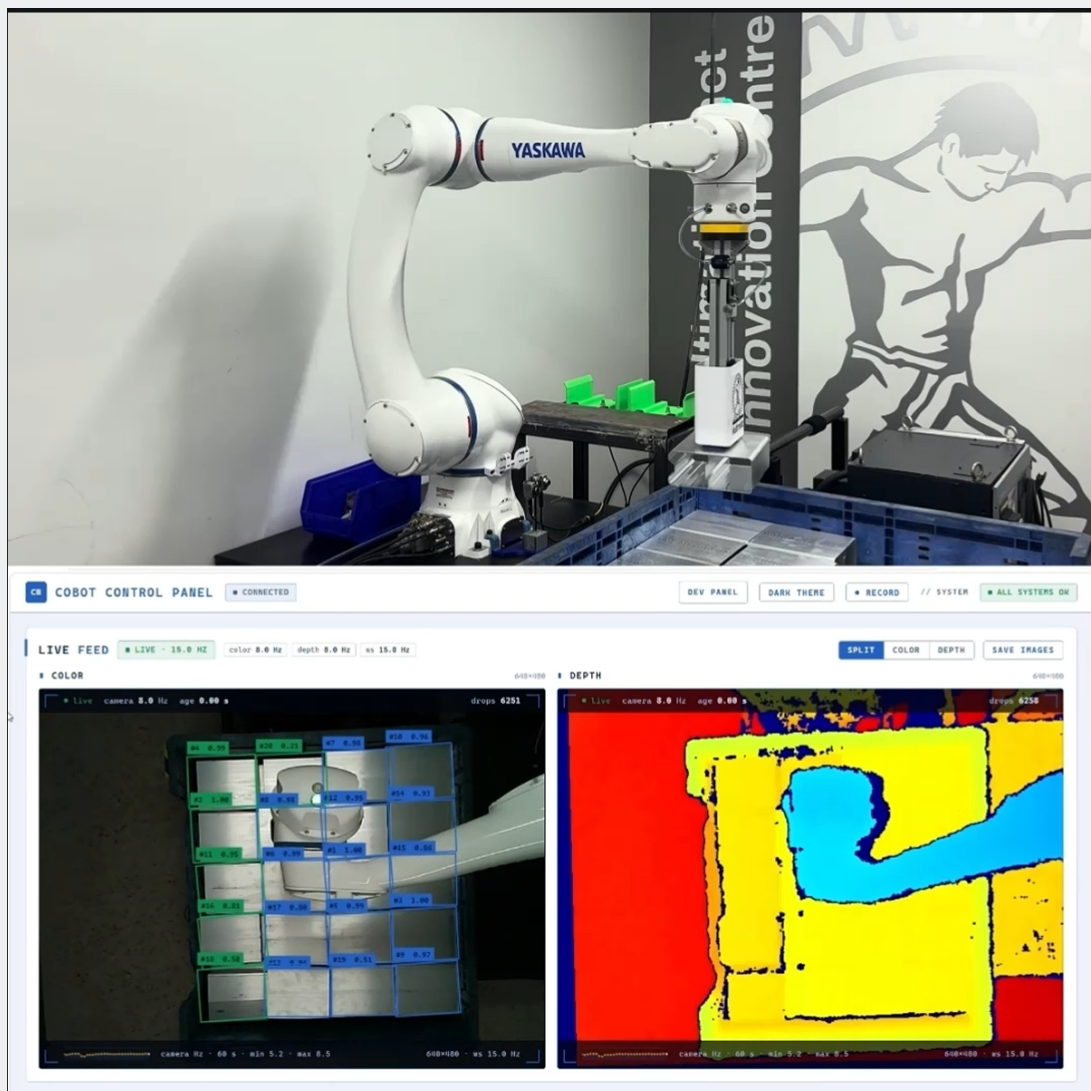
ROS based application integrating a Colaborative Robot with Computer Vision system for an automated convayer loading application

DATES May 2026 – Aug 2026

TAGS Manufacturing Automation, Computer Vision

TOOLS ROS2, C++, Python, ONNX, Ultralytics YOLO

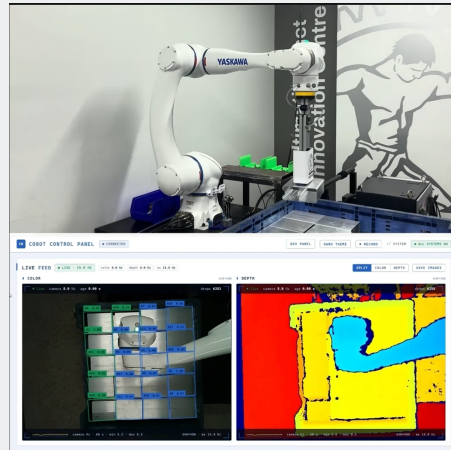
A ROS2 application for manipulating a Yaskawa Motoman Colaborative Robot, streaming from a RGB-D camera, and integrating the two with a YOLO based CV detector.



Robot and Live Dashboard

Pick and Place application live dashboard with RGB and Depth streams, and part bounding boxes.

This project included an entire ROS stack that bypassed the Cobot's pendant based UI and interfaced directly with its controller over a LAN connection, and an integrated a camera streamer, a part detection model, and an automatic calibration sequence. All these features composed a modular robotics system not just a singular automated process.

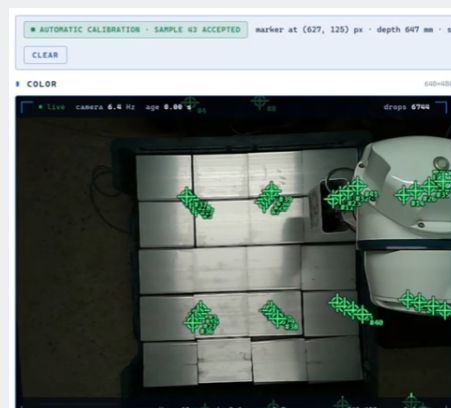


[Live Demo with Dashboard](#)

Robot Detecting and Pick/Placing one layer of parts into a fixture with <10mm positional tolerance.

[View on website](#)

One application of the system was to load parts into a convayer fixture with <10mm of tolerance. Firstly the custom trained detection model would be run to locate the each part on the top layer of the bin in depth camera coordinates. Then using the calibrated transformation matrix, their centerpoints and planar rotations would be transformed into the cobot's base coordinates. Finally the cobot would go to the center position, pick up the part, and load it into the fixture. The detection, coordinate transform and pick up action were all within the required tolerance so that the position of the fixure could be hard coded.



[Automatic Calibration Procedure](#)

An automated procedure for calculating the matrix transformation mapping camera frame coordinates to robot frame coordinates

[View on website](#)

PROJECT OUTCOME

The robot was able to load the fixture reliably for a layer of 20 parts. The detection model and camera calibration were able to transform coordinate systems within tolerance. The main software stack created a modular collaborative robot interface reusable for any future applications.

Humanoid Robot Operating System

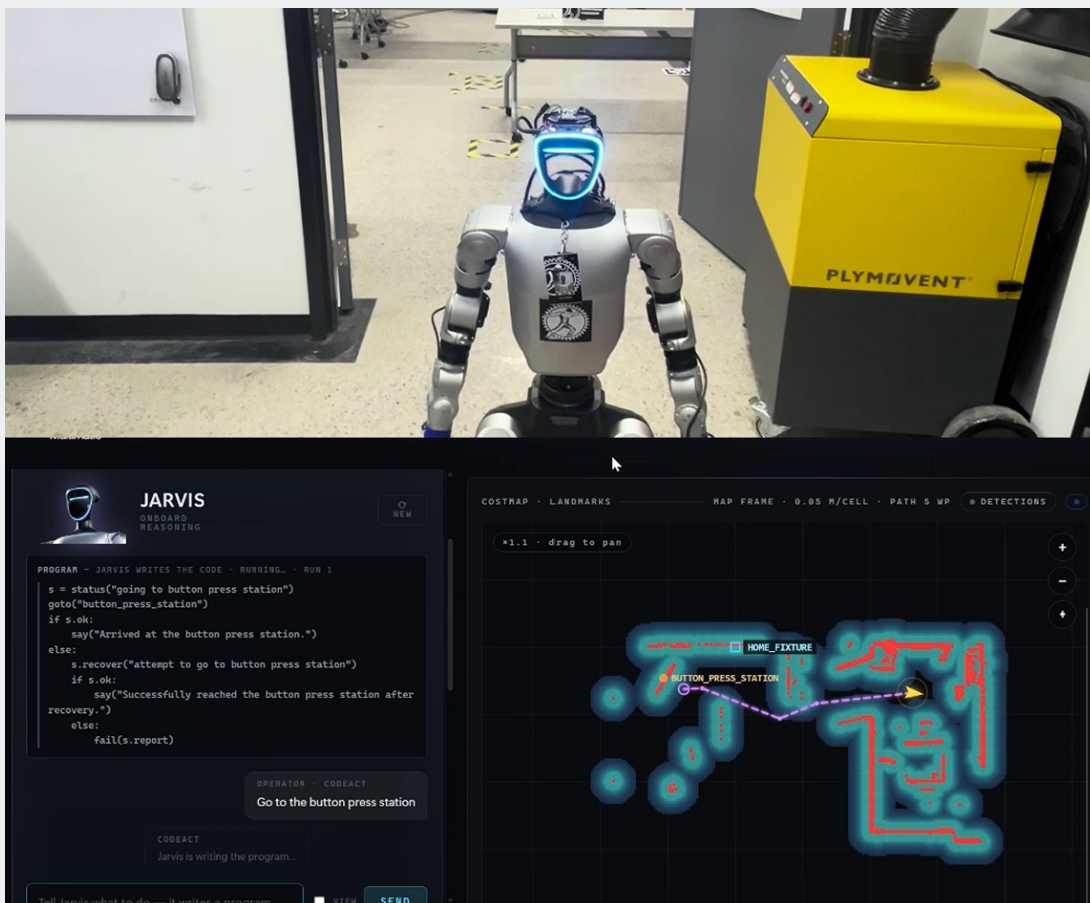
Lead software development for humanoid robot

DATES May 2026 – Aug 2026

TAGS Autonomus Humanoid Robotics, Reinforcement Learning, Real Time Operating System Design

TOOLS ROS2, NVIDIA IssacLab, C++, Python

A ROS2 based operating system for controlling a humanoid robot. Composing Computer Vision, SLAM/Odometry, VLM based Reasoning, and Trained RL policies into a deployable humanoid robot operating stack.



Humanoid Robot and Custom Operator Dashboard

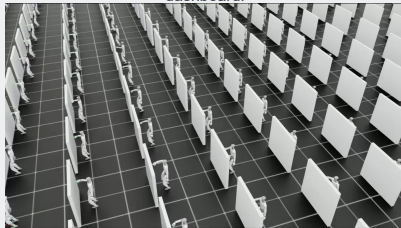
Still of the robot, and the dashboard with the reasoning (chatbot) interface, and the LiDAR costmap with a navigation path

CONCURRENT PROCESSES WITHIN THE HUMANOID OPERATING SYSTEM



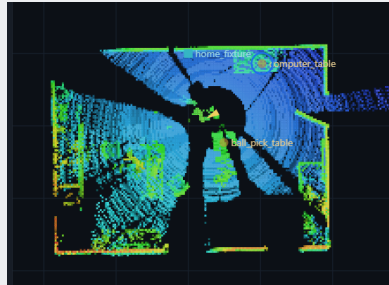
Reasoning Interface

Text based Chatbot interface for robot on web based dashboard.



Robots In IssacLab

Robots developing a RL policy for pressing a button.



Lidar Map

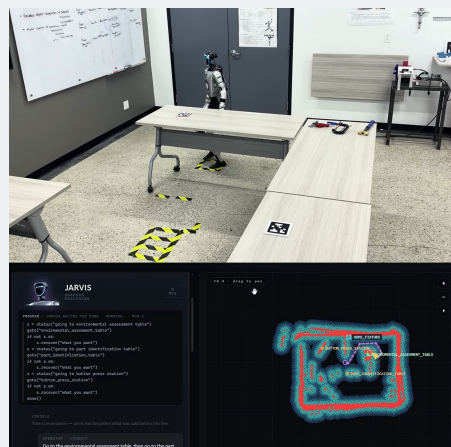
Lidar Pointcloud Projection with Remembered Landmarks.



Robot Perspective Vision

Live camera feed from the robot with live tracked detections.

The robot could work entirely autonomously, the local vision language model (A Lora fine tuned QWEN 3 model) could take text or voice based input, and use python syntax and a simple robot API to create dynamic workflows. This allowed the reactivity of artificial intelligence, and the predictability of typical code, perfect for a robot doing mostly repetitive tasks in a manufacturing setting. The robot API included functions that allowed the model to navigate wherever it chose, execute trained actions, inspect tracked objects, analyze its surroundings and react according to them. The model would also be able to handle failures, it had a recovery mode that would engage whenever one of the functions it called would return a failure, this prevented the robot from getting lost or trying to do an action that the robot could not preform.

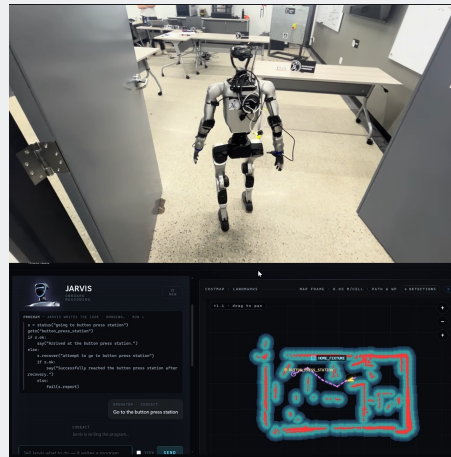


Three Station Navigation

The Robot planning a workflow to visit three workstations, based on a text prompt, then going to all three.

[View on website](#)

Navigation and Odometry used a two tiered SLAM + April Tag anchor system specifically designed for a robot on an industrial plant floor with multiple workstations to precisely navigate to. Data from the Robots LiDAR was projected into a 2D costmap which dynamically stored traversable and non traversable terrain, and remembered april tag landmarks. The mapping system allowed for the use of a traditional pathing algorithm (A*) for easy map traversal. The first tier of the navigation included following the A* traversal, with regular route validation and replans to react to a dynamic environment, to a dedicated remembered location where it could visibly see the april tag it was trying to navigate to. Once the desired tag was in view the robot could adjust itself by comparing its current 6D offset vector from the tag to the pre-captured offset vector using the tags ground truth. This created a simple, yet modular, interface for teaching the robot new navigation targets: capture the desired offset, show the tag, and the robot is able to autonomously able to navigate to the new workstation.



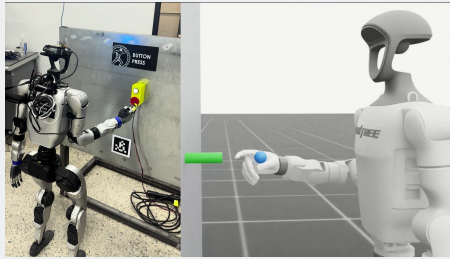
Obstacle Avoidance and Workstation Navigation

The robot navigating through doors, walking around tables, and docking to a specific offset from a remembered tag.

[View on website](#)

A two tiered perception system was used to join the speed of trained CV models with the intelligence of the onboard VLM. The design of the perception stack was geared towards part inspection, classification, and quality checks. Custom CV models were implemented using an ONNX runtime based detector. These raw bounding box + orientation detections fed through a tracking algorithm that turned raw, high frequency detections, into sustained object permanence. Using the camera's depth frame, tracked objects could be located in 3D space relative to the robot, and also in the world coordinate system using simple transforms. The other end of the perception stack was the high level reasoning model. By interfacing with the QWEN based VLM the robot could assess its surroundings, read instructions, and perform intelligent quality assessments on parts based on official specs. These quality assessments would receive only a crop of a bounding box detection and both text and images of what qualities of the part to judge. Composing these features together the robot could detect a specific part, locate it in 6D space, perform an assessment, and track the part holding onto its judgement.

Policies were developed in IsaacLab by designing complex reward systems and training curriculums. Once verified, the neural networks (ONNX) were transferred into the runtime stack, into a two tiered policy executor; high level policy control for permissions and transitions, and an ONNX runtime based policy runner. Policy execution used a forward kinematics engine, and the tracked detections to target objects, for example the button press.



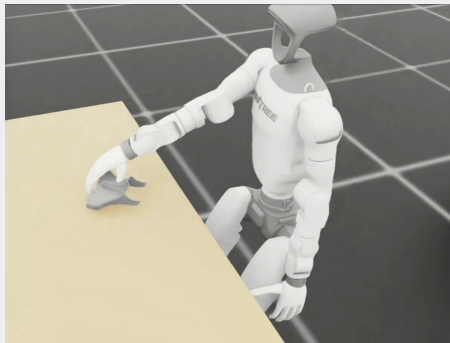
Robot Button Press

Reinforcement learning policy in NVIDIA IssacLab and deployed on hardware.

[View on website](#)

NEXT STEPS

Using IssacLab to develop a part pickup policy was the next step in development.



Reinforcement Learning Part Pickup

Robot Picking Up a Part in NVIDIA IssacLab developing a RL policy.

[View on website](#)

PROJECT OUTCOME

The humanoid's operating system had the ability to navigate to any remembered April tag position. The policy runner could run any RL policy dropped in from an ONNX file. Similarly, the tracker could track any detection class based on active detector ONNX files. The reasoning interface could carry out full conversations including describing the scene from the robot's perspective, and the robot API library was easily expandable for new robot abilities.

Design Team Experience

Controls Subsystem Member

Controls Firmware Developer

DATES Sep 2025 – May 2026

ORG University Of Waterloo Rocketry Team

Aided development of roll control canards for a supersonic rocket.

Internships

Advanced Manufacturing Intern

Automation Engineering

DATES May 2026 – Aug 2026

ORG Multimatic.

Programmed robots and developed software for researching and automating manufacturing processes.